

COMM116

Generative AI Applications

Section 1: Applications in Images and Videos

Dr Xiaoyang Wang

Department of Computer Science

x.wang7@exeter.ac.uk

Teaching Team



Module Conveners:

Dr Xiaoyang Wang, x.wang7@exeter.ac.uk

Dr Hang Dong, h.dong2@exeter.ac.uk

Module descriptor: COMM116

ELE page: <https://ele.exeter.ac.uk/course/view.php?id=27953>

About this module



Models, architectures, recent advances, and applications of generative AI.

We'd like to cover the following content:

Section 1 – Applications in Images and Videos

Section 2 – Language, Data and Knowledge

Section 3 - Addressing the Ethical Issues of LLMs

Section 4 – Autonomous Systems

Section 5 – Domain-Specific Applications

About this module



Two lectures per week, with one workshop (week 11 has no workshop; week 8 has an extended one.)

To understand recent advances - reading and discussing recent research papers;
Fast-changing domain; identify **trends**, discuss **plausibility**, **complexity**, constraints, **scalability**.

There is no single definitive textbook: we build understanding together through structured discussion, critical reading, and collaborative exploration.

Assessment

100% coursework; no exams. AI-assisted.

Coursework 1 Literature Review, Project Plan, and Presentation on a Generative AI Application (30%)

This assessment requires you to propose an application of generative AI. The submission will be

- A report covering literature review, project plan (including proposed methodology and evaluation plan) and feasibility study.
- A 5-minute presentation (in person).
- Slides for presentation, submitted at the same time as the report.



Assessment

Coursework 2 Implementation: Agentic Vision with Dynamic Tool Generation (70%).

This assessment requires you to build a **vision-centric AI agent** that answers visual questions.

The focus is on:

- agent orchestration (plan–act–observe–refine),
- dynamic tool/code generation,
- safe execution and reliability,
- evaluation and analysis.

You will not train large models; apply existing models.

Assessment

Coursework 2 Implementation: Agentic Vision with Dynamic Tool Generation (70%).

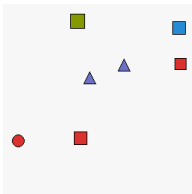
This assessment requires you to build a **vision-centric AI agent** that answers visual questions.

The focus is on:

- agent orchestration (plan–act–observe–refine),
- dynamic tool/code generation,
- safe execution and reliability,
- evaluation and analysis.

You will not train large models; apply existing models.

Here is an example. Given an image and a question, the AI agent provides a correct answer.



question: How many blue squares
are there? Answer with an
integer.
answer: 1

Assessment



Submission deadlines

Coursework 1 - 24/02/2026, 12:00 pm; submit a .zip file with **report** and **presentation slides** (.pdf).

In-person presentation: Wednesday, 4 March 2026, 9:35 - 11:25 - see timetable.

Coursework 2 - 26/03/2026, 12:00 pm; submit a .zip file containing **code**, **report** and a short **demo** video.

Recap: What is Generative AI?

Please go to: <https://www.menti.com/>

Code: 6657 4233

Or scan the QR code:



Recap: What is Generative AI?

- **Generative AI** builds a model of a *data distribution* and then *samples* from it.
- Data: images, text, audio, videos, molecules, ...
- Goal: generate new samples that “look like” the training data, while being *novel* (not memorized; not exactly the same with the training data).

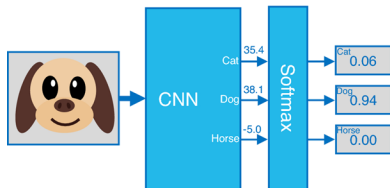
Abstract Model

$$x \sim p_{\text{data}}(x) \quad \implies \quad \text{learn } p_{\theta}(x) \approx p_{\text{data}}(x) \quad \text{and sample } \tilde{x} \sim p_{\theta}(x).$$

Discriminative vs Generative Models

Discriminative Model

$p(y|x) \rightarrow$ probability mass over **labels***



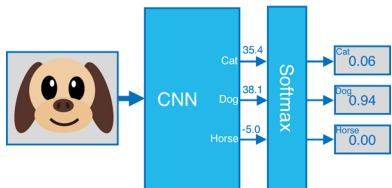
* *discriminative model $\neq$ discrete outputs*

figure from: Cardarilli, G.C., Di Nunzio, L., Fazzolari, R., Giardino, D., Nannarelli, A., Re, M. and Spanò, S., 2021. A pseudo-softmax function for hardware-based high speed image classification. Scientific reports, 11(1). p.15307.

Discriminative vs Generative Models

Discriminative Model

$p(y|x) \rightarrow$ probability mass over **labels***



* *discriminative model $\neq$ discrete outputs*

Generative Model

$p(x) \rightarrow$ probability density of samples*

* *If x is discrete, $p(x)$: probability mass function*

Conditional Generative Model

$p(x|y) \rightarrow$ leads to some paradigm change of ML tasks!



figure from: Cardarilli, G.C., Di Nunzio, L., Fazzolari, R., Giardino, D., Nannarelli, A., Re, M. and Spanò, S., 2021. A pseudo-softmax function for hardware-based high speed image classification. Scientific reports, 11(1). p.15307.

Generative Models Can

Learn features in an unsupervised manner.

Sample to generate new data from the same distribution.

Detect low likelihood outliers (with a trained model).

...

Why distributions matter

If you can model $p(x)$, you can: sample, impute missing data, detect anomalies, compress, and do Bayesian updates.

Distributional View

Learning a distribution

We never observe $p_{\text{data}}(x)$ directly; only a dataset $\mathcal{D} = \{x_i\}_{i=1}^N$.

Example: Let x be a sentence (a sequence of tokens).

$p_{\text{data}}(x)$ would mean: “How likely is this exact sentence to appear in real human text?”

We can't compute it directly. We only have a corpus:

$$\mathcal{D} = \{x_i\}_{i=1}^N,$$

e.g., millions of sentences from books/web pages.

A language model learns $p_{\theta}(x)$ from these examples—without ever knowing the true $p_{\text{data}}(x)$.

Learning a distribution

Two common training views (training objectives)

- **Likelihood-based:** given a dataset $\mathcal{D} = \{x_i\}_{i=1}^N$, maximize the log-likelihood

$$\max_{\theta} \sum_{x_i \in \mathcal{D}} \log p_{\theta}(x_i) \quad \left(= \max_{\theta} \sum_{i=1}^N \log p_{\theta}(x_i) \right).$$

- There are variations of the likelihood-based approaches, such as VAE - likelihood-based in the variational sense.
- **Simulation-based / implicit:** train a sampler $x = g_{\theta}(\epsilon)$ so samples match data; the model is defined by its sampling process, not by a closed-form probability.

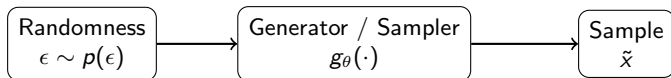
Sampling is the end goal

A generative model is only as good as the *samples* it can produce:

$\tilde{x} \sim p_{\theta}(x)$ should be high-quality, diverse, and data-like.

Sampling

Many generative models define not just a distribution, but a *recipe* to draw samples (with a trained generative model).



- Autoregressive models: sample token-by-token.
- VAEs: sample latent z , then decode.
- Diffusion models: start from noise, iteratively denoise.

Sampling with a known distribution



Sampling with a known distribution

Example: sampling from an Exponential distribution via inverse CDF.

Suppose we want to sample a waiting time x from an Exponential distribution

$$p(x) = \lambda e^{-\lambda x}, \quad x \geq 0.$$

Step 1 (uniform randomness). A computer pseudorandom number generator (PRNG, link) generates

$$u \sim \mathcal{U}(0, 1).$$



Sampling with a known distribution

Example: sampling from an Exponential distribution via inverse CDF.

Suppose we want to sample a waiting time x from an Exponential distribution

$$p(x) = \lambda e^{-\lambda x}, \quad x \geq 0.$$

Step 1 (uniform randomness). A computer pseudorandom number generator (PRNG, link) generates

$$u \sim \mathcal{U}(0, 1).$$

Step 2 (inverse CDF transform). The CDF is

$$F(x) = \Pr(X \leq x) = 1 - e^{-\lambda x}.$$

Set $u = F(x)$ and solve for x :

$$u = 1 - e^{-\lambda x} \Rightarrow e^{-\lambda x} = 1 - u \Rightarrow x = -\frac{1}{\lambda} \ln(1 - u).$$

Therefore, an exact sample is obtained by

$$x = F^{-1}(u) = -\frac{1}{\lambda} \ln(1 - u), \quad u \sim \mathcal{U}(0, 1).$$



Sampling in (deep) generative models

For real data (images/audio/text/...), x is high-dimensional.

- Example (image): $x \in \mathbb{R}^{256 \times 256 \times 3} \Rightarrow \sim 2 \times 10^5$ dimensions.
- There is no practical notion of a global CDF $F(x)$ in high dimensions, and no tractable inverse F^{-1} .

Sampling in (deep) generative models

For real data (images/audio/text/...), x is high-dimensional.

- Example (image): $x \in \mathbb{R}^{256 \times 256 \times 3} \Rightarrow \sim 2 \times 10^5$ dimensions.
- There is no practical notion of a global CDF $F(x)$ in high dimensions, and no tractable inverse F^{-1} .

Even if we can score a sample, sampling can still be hard.

- We might know an *unnormalized* density $\tilde{p}(x) \propto p(x)$, but not the normalizing constant. In high dimensions, computing the normalizing constant is usually intractable.

$$Z = \int \tilde{p}(x) dx$$

- Or we may not have any explicit $p_\theta(x)$ at all (implicit models): we only know how to *generate* samples.

Sampling in (deep) generative models

For real data (images/audio/text/...), x is high-dimensional.

- Example (image): $x \in \mathbb{R}^{256 \times 256 \times 3} \Rightarrow \sim 2 \times 10^5$ dimensions.
- There is no practical notion of a global CDF $F(x)$ in high dimensions, and no tractable inverse F^{-1} .

Even if we can score a sample, sampling can still be hard.

- We might know an *unnormalized* density $\tilde{p}(x) \propto p(x)$, but not the normalizing constant. In high dimensions, computing the normalizing constant is usually intractable.

$$Z = \int \tilde{p}(x) dx$$

- Or we may not have any explicit $p_\theta(x)$ at all (implicit models): we only know how to *generate* samples.

Intuition: “turning random bits into (generating) a realistic image” is a highly structured transformation; not in closed-form (intractable).

Sampling in (deep) generative models

Idea:

Instead of inverting a CDF, we design model families where sampling is *procedural*:

- **Autoregressive:** factorize and sample sequentially

$$p_{\theta}(x) = \prod_{t=1}^T p_{\theta}(x_t \mid x_{<t}) \quad \Rightarrow \quad x_1 \rightarrow x_2 \rightarrow \cdots \rightarrow x_T.$$

Sampling in (deep) generative models

Idea:

Instead of inverting a CDF, we design model families where sampling is *procedural*:

- **Autoregressive:** factorize and sample sequentially

$$p_{\theta}(x) = \prod_{t=1}^T p_{\theta}(x_t \mid x_{<t}) \quad \Rightarrow \quad x_1 \rightarrow x_2 \rightarrow \cdots \rightarrow x_T.$$

- **Latent-variable (VAE-style):** sample simple latent then decode

$$z \sim p(z), \quad x \sim p_{\theta}(x \mid z).$$



Sampling in (deep) generative models

- **Normalizing flows:** learn an invertible transform

$$\epsilon \sim \mathcal{N}(0, I), \quad x = f_{\theta}(\epsilon).$$

Sampling in (deep) generative models

- **Normalizing flows:** learn an invertible transform

$$\epsilon \sim \mathcal{N}(0, I), \quad x = f_{\theta}(\epsilon).$$

- **Diffusion:** start from noise and iteratively denoise

$$x_T \sim \mathcal{N}(0, I), \quad x_T \rightarrow x_{T-1} \rightarrow \cdots \rightarrow x_0.$$

Key conceptual shift: we replace “closed-form sampling” with *structured computation* (sequential steps, latent decoding, or iterative refinement).

For discriminative models such as classifiers, the trained model is ready for inference (make a decision); for generative models, trained model + sampling procedure are presented together.

Conditional generation

Often we want $p_{\theta}(x | c)$ where c is a condition:

- class label, text prompt, style, segmentation map, ...
- **Modeling:** learn the conditional distribution $p_{\theta}(x | c)$; paired training data $(x_i, c_i)^*$
- **Sampling:** produce $\tilde{x}$ that matches c while staying realistic

** paired training data is not always necessary. For example, For diffusion, we can train an unconditional model and then “steer” samples in the sampling process. c is not given during training.*

Consider

Fidelity (high quality) vs. **diversity** (many modes) vs. **controllability** (follows c).



Evaluating samples

We typically evaluate on:

- **Fidelity:** Are samples realistic?
- **Diversity:** Do we cover many modes (no collapse)?
- **Alignment (conditional):** Does $\tilde{x}$ match the condition/prompt?
- **Efficiency:** training cost, sampling cost (latency), memory

Important framing for diffusion

In diffusion, *sampling cost* is dominated by the number of denoising steps $\times$ backbone compute
- will be discussed in DiT.

Milestones of (Modern) GenAI Applications

- **2013: Variational Autoencoders (VAEs)**

- Kingma, D.P. and Welling, M., 2013. Auto-encoding variational bayes. arXiv preprint arXiv:1312.6114.

- **2014: Generative Adversarial Networks (GANs)**

- Goodfellow, I.J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. and Bengio, Y., 2014. Generative adversarial nets. Advances in neural information processing systems, 27.

- **2017: Transformer architecture**

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. and Polosukhin, I., 2017. Attention is all you need. Advances in neural information processing systems, 30.

- **From 2007: Hardware breakthrough**

- CUDA was created by Nvidia starting in 2004 and was officially released in 2007.
- Allows software to use certain types of graphics processing units (GPUs) for accelerated general-purpose processing.

Milestones of (Modern) GenAI Applications

The GPT series

- **2018: GPT-1, OpenAI**

- Radford, A., Narasimhan, K., Salimans, T. and Sutskever, I., 2018. Improving language understanding by generative pre-training.

- **2019: GPT-2, OpenAI**

- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D. and Sutskever, I., 2019. Language models are unsupervised multitask learners. OpenAI blog, 1(8), p.9.
- 1.5-billion-parameter model, trained on an 8-million-web-page dataset.

- **2020: GPT-3, OpenAI**

- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A. and Agarwal, S., 2020. Language models are few-shot learners. Advances in neural information processing systems, 33, pp.1877-1901.
- 75 billion parameters model

Milestones of (Modern) GenAI Applications

The GPT series (continued)

- **2022: GPT-3.5, OpenAI**

- A sub class of GPT-3 Models.
- From GPT-3, model parameters were not released.
- chatGPT released.

- **2023: GPT-4, OpenAI**

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F.L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S. and Avila, R., 2023. GPT-4 technical report. arXiv preprint arXiv:2303.08774.
- Multimodal.

- **2025: GPT-5, OpenAI**

- Multimodal.



Milestones of (Modern) GenAI Applications

Image and video generation

- **2021: DALL-E, OpenAI**

- Text to image model.
- Used a modified version of GPT-3.
- 2022, DALL-E 2
- 2023, DALL-E 3 (released natively into ChatGPT)
- 2025, replaced by GPT Image

- **2022: Stable Diffusion, Stability AI**

- Originated from a project called Latent Diffusion.
- CLIP (Contrastive Language–Image Pre-training) text encoder.
- Optimised for customer use - an optimised version requires 2.4GB VRAM.

Application types

Chat assistants: general-purpose Q&A, writing, planning, tutoring.

- Example: ChatGPT (public release, 2022).

Code copilots: autocomplete, refactor, tests, documentation, code review.

- Example: GitHub Copilot (general availability, 2022); Cursor (developed by Anysphere; latest release is Cursor 2.0).

Workplace copilots: drafting, meeting notes, spreadsheets, slide generation.

- Example: Microsoft 365 Copilot (announced 2023).

Video / audio generation: short clips, b-roll, storyboards, music/sound effect.

- Examples: Runway Gen-2 (2023), OpenAI Sora preview (2024) and Sora 2 (2025).

Enterprise solution ● Gemini Enterprise, Oct 2025; <https://cloud.google.com/blog/products/ai-machine-learning/introducing-gemini-enterprise>

Summary

Takeaway

Generative AI = **learn** a distribution $p_{\theta}(x)$ (or $p_{\theta}(x | c)$) and **sample** from it.

Next:

- a brief recap of **diffusion** and its sampling procedure.
- U-Net.
- **Transformers** and DiT.

Week 1 workshop:

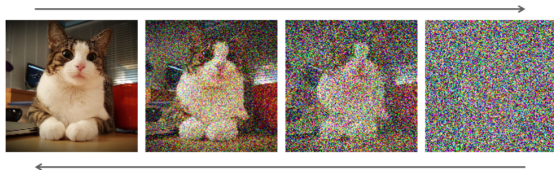
- Sampling with known distributions.

DDPM

Ho, J., Jain, A. and Abbeel, P., 2020. Denoising diffusion probabilistic models. Advances in neural information processing systems, 33, pp.6840-6851.

Diffusion models

- Data are samples from an unknown distribution $p_{\text{data}}(x)$ (e.g., images).
- A generative model learns $p_{\theta}(x)$ such that we can sample $x \sim p_{\theta}$.
- Diffusion: generate by **iteratively denoising** from Gaussian noise to data.



From left to right: forward process; $x_0 \rightarrow x_T$
Right to left: reverse (denoising) process; $x_T \rightarrow x_0$

Forward process

Gradually add Gaussian noise.

- Define a Markov chain that corrupts data:

$$q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t} x_{t-1}, (1 - \alpha_t)I),$$

with $\alpha_t \in (0, 1)$.

- After many steps, $x_T \approx \mathcal{N}(0, I)$.

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I),$$

$$\bar{\alpha}_t = \prod_{i=1}^t \alpha_i.$$



Reverse process

Learn to denoise step-by-step.

- We want the reverse-time transitions $p_\theta(x_{t-1} \mid x_t)$.
- Parameterize as Gaussian (if α_t is close to 1):

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(\mu_\theta(x_t, t), \Sigma_\theta(x_t, t)).$$

- Sampling: start $x_T \sim \mathcal{N}(0, I)$, then iterate $t = T, \dots, 1$.

Training objective: predict the noise (DDPM)

- Common parameterization: predict ε used in forward process.
- Loss (simplified form):

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{t, x_0, \varepsilon} \left[\|\varepsilon - \varepsilon_{\theta}(x_t, t)\|_2^2 \right].$$

- Intuition: if we can estimate the noise, we can denoise.

See DDPM: Ho, J., Jain, A. and Abbeel, P., 2020. Denoising diffusion probabilistic models. Advances in neural information processing systems, 33, pp.6840-6851.



DDPM training loop

- Inputs: dataset $\{x_0\}$, noise schedule $\{\alpha_t\}_{t=1}^T$ with $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$, network $\varepsilon_\theta(\cdot, t)$.

Repeat for each optimization step

- 1 Sample a minibatch of clean data $x_0 \sim p_{\text{data}}$.
- 2 Sample timesteps $t \sim \text{Uniform}(\{1, \dots, T\})$ (one t per example).
- 3 Sample noise $\varepsilon \sim \mathcal{N}(0, I)$.
- 4 Form noisy inputs (closed form):

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon.$$

- 5 Predict noise: $\hat{\varepsilon} \leftarrow \varepsilon_\theta(x_t, t)$.
- 6 Compute loss (MSE):

$$\mathcal{L} = \frac{1}{B} \sum_{i=1}^B \left\| \varepsilon^{(i)} - \hat{\varepsilon}^{(i)} \right\|_2^2.$$

- 7 Gradient step: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$.

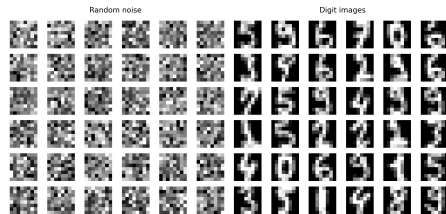
Image distributions

Take a step back to look at image distributions.

Image distributions

Take a step back to look at image distributions.

- An image is a point in a high-dimensional space: $x \in \mathbb{R}^{H \times W}$.
- **Random noise** explores this space broadly.
- **Real images** concentrate on a tiny subset: they follow $p_{\text{data}}(x)$.
- Generative modeling = learn $p_{\theta}(x) \approx p_{\text{data}}(x)$ and sample from it.

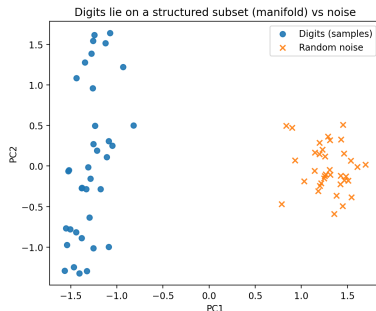


Left: random noise samples Right: digit samples

Distribution view

Digits occupy a structured region of pixel space (on or near a low-dimensional manifold).

- Flatten each image into a vector (pixel space).
- Visualize with PCA (2D projection).
- Digits cluster / form structure; noise is spread differently.
- This motivates diffusion: start from simple noise distribution, then “move” toward data.



Illustration; not real data.

Diffusion models

Diffusion as a *path of probability distributions*

- Let $q_0(x) := p_{\text{data}}(x)$ be the (unknown) data distribution.
- Define a forward Markov chain that adds Gaussian noise:

$$q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t} x_{t-1}, (1 - \alpha_t)I).$$

- This induces a sequence of *marginals* (distributions over images at noise level t):

$$q_t(x) := \int q(x_t = x \mid x_0) q_0(x_0) dx_0.$$

- Intuition: q_t is p_{data} after being *progressively smoothed* by Gaussian noise.
- For large t , the marginal becomes simple:

$$q_T(x) \approx \mathcal{N}(0, I).$$

Generation/Sampling

- The forward chain builds:

$$q_0 \rightarrow q_1 \rightarrow \cdots \rightarrow q_T \approx \mathcal{N}(0, I).$$

- To sample from q_0 , start from the easy distribution $x_T \sim \mathcal{N}(0, I)$, then run a learned reverse chain:

$$p_\theta(x_{t-1} \mid x_t).$$

- Goal: make reverse marginals track the forward marginals:

$$p_{\theta,t}(x) \approx q_t(x) \implies p_{\theta,0}(x) \approx q_0(x) = p_{\text{data}}(x).$$

- Diffusion learns a sequence of *small, local moves* that transport mass from a simple base distribution to a complex target distribution; in a **controlled** way.

Sampling

- The **score** (vector field) of the noisy marginal is

$$\nabla_x \log q_t(x),$$

which points toward regions of higher probability under q_t .

- Reverse-time dynamics (in continuous time) depend on this score:

$$\text{reverse drift} \propto \nabla_x \log q_t(x).$$

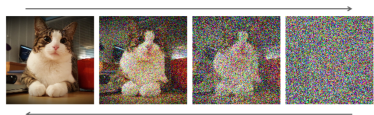
- Learning $\varepsilon_\theta(x_t, t)$ is a practical way to learn this score *implicitly*.
- Sampling = repeated score-driven refinement: gradually concentrate probability mass back onto data-like images.

Latent diffusion Model

Latent diffusion



Latent diffusion



- Pixel diffusion operates on $x \in \mathbb{R}^{H \times W \times C}$ (high-dimensional).
- Input and output should have the same dimension
- U-Nets / attention over large feature maps $\Rightarrow$ heavy compute + memory.
- Observation: much of the pixel space is “redundant” (images live on a structured subset/low-dimensional manifold).
- Idea: **compress** images to a latent space, run diffusion there.

Latent Diffusion

- Train an encoder–decoder (autoencoder):

$$z = E(x), \quad \hat{x} = D(z).$$

- Latent z has much smaller spatial resolution (e.g., $h \times w \ll H \times W$).
- Train a diffusion model on latents z instead of pixels x .

Key benefit

Same diffusion approach, but on a lower-dimensional representation $\Rightarrow$ much faster.

Latent Diffusion

- Treat $z_0 = E(x)$ as “data” for diffusion.
- Forward noising:

$$z_t = \sqrt{\bar{\alpha}_t} z_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I).$$

- Train a denoiser (often still a U-Net, but smaller):

$$\mathcal{L}(\theta) = \mathbb{E}_{t, z_0, \varepsilon} \left[\|\varepsilon - \varepsilon_\theta(z_t, t)\|_2^2 \right].$$

Sampling

- ① Sample latent noise: $z_T \sim \mathcal{N}(0, I)$.
- ② Run reverse diffusion to obtain z_0 .
- ③ Decode to pixel space: $\hat{x} = D(z_0)$.

Diffusion learns $p(z)$ in a compact space; the decoder maps latents back to images.

Efficiency and scalability

- Pixel diffusion complexity scales with $H \times W$.
- Latent diffusion scales with $h \times w$, where typically h, w are smaller by a factor (e.g., 8).
- Roughly: spatial compute drops by $(HW)/(hw)$ (often $\sim 64\times$).
- This makes high-res generation and attention-based conditioning feasible.

From DDPM to LDM



The idea of latent diffusion sounds straightforward; operating in latent space is not uncommon. Why not in the original DDPM?

From DDPM to LDM

The idea of latent diffusion sounds straightforward; operating in latent space is not uncommon. Why not in the original DDPM?

- DDPM (Ho et al., 2020) established diffusion as a strong pixel-space generative model with a clean mathematical likelihood/ELBO formulation and competitive sample quality.
- Latent Diffusion (Rombach et al., 2022) is a later systems/engineering step: compress with an autoencoder, then diffuse in the latent space.

Primary goal

- DDPM's goal: **present a framework that works** with a simple, principled setup.
- Pixel space keeps the story tight:
 - works directly on $p(x)$
 - forward process is known and simple on x ,
 - reverse model directly defines $p_\theta(x_0)$,
 - ELBO decomposition is straightforward to present and implement.
- Latent diffusion adds an extra learned module: E, D and additional design choices.

Adding an autoencoder

Adding an autoencoder introduces new failure modes.

- Latent diffusion depends on the **representation quality** of the autoencoder:

$$z = E(x), \quad \hat{x} = D(z).$$

- If E, D discard details or create artifacts, diffusion in z cannot recover them.
- So LDM uses a two-stage training pipeline (while DDPM is **end-to-end**):
 - 1 Train a strong perceptual compression model.
 - 2 Train diffusion on its latent space.
- DDPM avoided this extra failure mode to keep results attributable to diffusion itself.

Growing need for complex conditioning

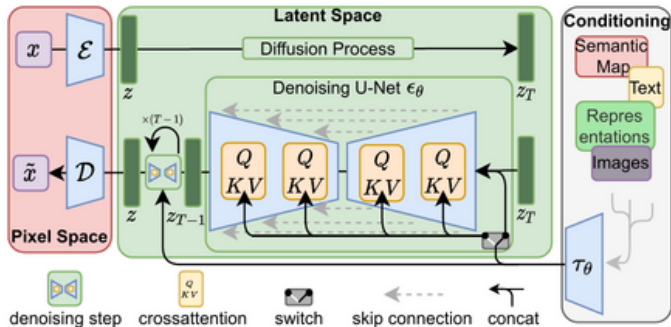
- Pixel-space diffusion is expensive; latent-space diffusion is far cheaper.
- In DDPM, experiments are on CIFAR10 (32×32) and CelebA-HQ (256×256).
- Main wins unlocked by running diffusion on $h \times w \ll H \times W$:
 - **High-resolution generation** becomes feasible within compute budgets. In the original LDM paper, the generated image can go to $\sim 1024^2$ px.
 - **Attention-based conditioning** (e.g., cross-attention to text tokens) becomes practical.
- These became central once the community pushed hard on **high-res** and **general conditioning** (e.g., text-to-image).

Link: Latent diffusion

LDM



University
of Exeter



Condition LDMs either via concatenation or by a more general cross-attention mechanism.

Tradeoffs

- **Autoencoder bottleneck:** if E, D lose details, diffusion cannot recover them.
- **Artifacts:** compression can introduce texture bias or decoding artifacts.
- **Two-stage training:** must train a good latent space *first*.
- Still, the speed/quality tradeoff is excellent in practice.

Recommended Reading for Diffusion

<https://lilianweng.github.io/posts/2021-07-11-diffusion-models/>

Luo, C., 2022. Understanding diffusion models: A unified perspective. arXiv preprint arXiv:2208.11970.

Latent diffusion:

Rombach, R., Blattmann, A., Lorenz, D., Esser, P. and Ommer, B., 2022. High-resolution image synthesis with latent diffusion models. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (pp. 10684-10695).

For conditional generation:

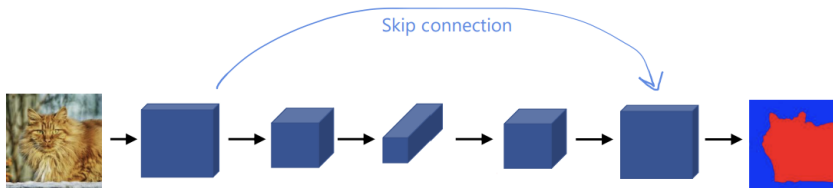
Ho, J. and Salimans, T., 2022. Classifier-free diffusion guidance. arXiv preprint arXiv:2207.12598.

Architectures

The default backbone: U-Net

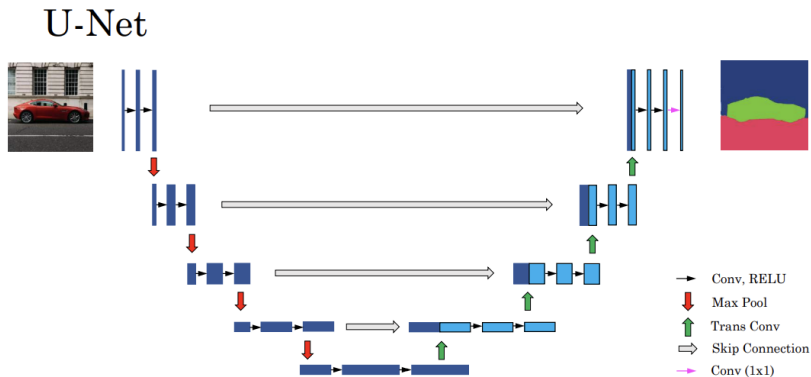
- Diffusion needs a network $f_{\theta}(x_t, t)$ that works across *all noise levels* t .
- U-Net is a strong image-to-image architecture with the right inductive biases:
 - **Multi-scale** processing (coarse semantics + fine details).
 - **Skip connections** preserve low-level spatial information for reconstruction.
 - **Fully convolutional**.
 - Same spatial dimension for input and output.

Deep Learning for Semantic Segmentation



Andrew Ng

U-Net Structure



[Ronneberger et al., 2015, U-Net: Convolutional Networks for Biomedical Image Segmentation]

Andrew Ng

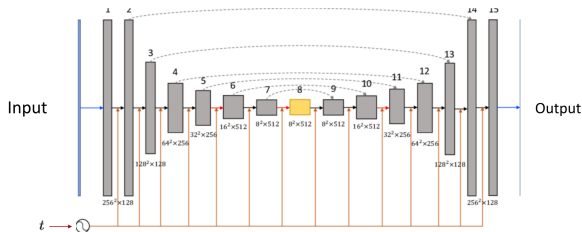
U-Net in diffusion

- Input: noisy image x_t and **timestep** t (and optional condition c).
- Output target depends on parameterization:
 - ε -prediction: $\hat{\varepsilon} = \varepsilon_\theta(x_t, t)$.
 - x_0 -prediction: $\hat{x}_0 = x_{0,\theta}(x_t, t)$.
 - predicting v : $\hat{v} = v_\theta(x_t, t)$ (common in practice)*.
- Network learns “how to denoise” at noise level t .

* See Salimans, T. and Ho, J., 2022. *Progressive distillation for fast sampling of diffusion models*. *arXiv preprint arXiv:2202.00512*.

U-Net in diffusion

- Timestep embedding: $e_t = \text{Embed}(t)$ (often sinusoidal + MLP).
- Adding e_t throughout the U-Net:
 - The simplest way is to add to activations; this is called modulating the network across layers.



Attention in U-Nets

- Pure convolutional networks have local receptive fields; attention adds global interactions.
- Common design: add self-attention blocks at intermediate resolutions (e.g., 16×16 , 32×32).
- For text conditioning: cross-attention in the decoder/blocks.

Result

U-Net + (some) attention = strong baseline for image diffusion.

Strengths and limitations

Strengths

- Inductive bias for images.
- Multi-scale reconstruction is natural.
- Efficient and stable to train.

Limitations

- Scaling to very large capacity can be less “standardized” than Transformers.
- Global interactions rely on attention add-ons.
- Conditioning and modality-generalization can become architecture-specific.

- As we scale data/compute, we want architectures that:
 - scale predictably with model size,
 - integrate conditioning (text, class, layout) cleanly,
 - generalize beyond images (video, 3D, multimodal),
 - benefit from mature Transformer tooling (pretraining, optimization, parallelism).

Transformers as a scalable backbone

- Transformer with vision applications: Vision Transformers (ViT) treat an image as a sequence of patch tokens.
- Self-attention gives global receptive fields from the start.
- Transformers have strong scaling behavior in many domains (NLP, vision, multimodal).
- Natural interfaces for conditioning (cross-attention, tokens, prefixes).

ViT

ViT idea: treat an image as a **sequence of patch tokens** and apply a standard Transformer encoder.

Dosovitskiy, A., 2020. An image is worth 16x16 words: Transformers for image recognition at scale. arXiv preprint arXiv:2010.11929. ICLR 2021.

ViT

ViT idea: treat an image as a **sequence of patch tokens** and apply a standard Transformer encoder.

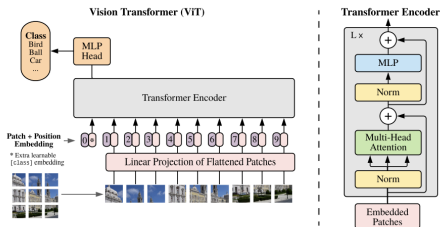


Figure 1: Model overview. We split an image into fixed-size patches, linearly embed each of them, add position embeddings, and feed the resulting sequence of vectors to a standard Transformer encoder. In order to perform classification, we use the standard approach of adding an extra learnable “classification token” to the sequence. The illustration of the Transformer encoder was inspired by Vaswani et al. (2017).

Dosovitskiy, A., 2020. An image is worth 16x16 words: Transformers for image recognition at scale. arXiv preprint arXiv:2010.11929. ICLR 2021.



Patchify: image $\rightarrow$ tokens

Understand the dimension.

- Input image: $x \in \mathbb{R}^{H \times W \times C}$.
- Choose patch size $P \times P$ (assume H, W divisible by P).
- Number of patches (tokens):

$$N = \frac{H}{P} \cdot \frac{W}{P}.$$

- Flatten each patch to a vector in $\mathbb{R}^{P^2 C}$, then linearly project to model width D :

$$\text{patch}_i \in \mathbb{R}^{P^2 C}, \quad z_i = W_E \text{patch}_i \in \mathbb{R}^D.$$

- Stack tokens: $Z \in \mathbb{R}^{N \times D}$ (or $B \times N \times D$ with batch).



Positional Encoding

- Transformers are permutation-invariant $\Rightarrow$ need positions.
- Add learned or sinusoidal positional embeddings:

$$Z_0 = Z + E_{\text{pos}}, \quad E_{\text{pos}} \in \mathbb{R}^{N \times D}.$$

- For classification, ViT often prepends a learnable $[CLS]$ token:

$$Z_0 = [z_{\text{cls}}; Z] + E_{\text{pos}}, \quad Z_0 \in \mathbb{R}^{(N+1) \times D}.$$

Transformer encoder block (ViT)

- Apply L standard encoder blocks:

$$Z_{\ell+1} = \text{Block}(Z_{\ell}), \quad \ell = 0, \dots, L-1.$$

- Each block (Pre-LN form):

$$\tilde{Z} = Z + \text{MHA}(\text{LN}(Z)), \quad Z' = \tilde{Z} + \text{MLP}(\text{LN}(\tilde{Z})).$$

- Output token features encode both local and global image content.

Layer Norm

LayerNorm for a single token vector $h \in \mathbb{R}^D$

$$\text{LN}(h) = \gamma \odot \frac{h - \mu(h)}{\sqrt{\sigma(h)^2 + \epsilon}} + \beta, \quad \mu = \frac{1}{D} \sum_{i=1}^D h_i, \quad \sigma^2 = \frac{1}{D} \sum_{i=1}^D (h_i - \mu)^2,$$

$\gamma, \beta \in \mathbb{R}^D$, $\epsilon > 0$, $\odot$ denotes elementwise multiplication.

- Normalises each token's feature vector to have mean 0 and variance 1 (then applies learned scale/shift).
- LN is applied independently to each token (each (b,n) vector of length D).

Multi-head self-attention

Let $Z \in \mathbb{R}^{B \times N \times D}$, number of heads h , head dim $d = D/h$.

- Linear projections:

$$Q = ZW_Q, K = ZW_K, V = ZW_V \in \mathbb{R}^{B \times N \times D}.$$

- Reshape to heads: $Q, K, V \in \mathbb{R}^{B \times h \times N \times d}$.
- Attention weights per head:

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) \in \mathbb{R}^{B \times h \times N \times N}.$$

- Output:

$$\text{MHA}(Z) = \text{Concat}(AV)W_O \in \mathbb{R}^{B \times N \times D}.$$

Patch size in ViT

- Self-attention's computation cost scales as $O(N^2D)$; N is the sequence length.
- $N = \frac{H}{P} \frac{W}{P} \Rightarrow$ smaller patches (smaller P) $\Rightarrow$ larger $N \Rightarrow$ more expensive attention.
- Tradeoff:
 - small P : better fine detail, higher compute.
 - large P : cheaper, but may lose local detail unless model is strong.

Classification

After patchifying and running L Transformer encoder blocks, we obtain the final token embeddings:

- patch tokens: $z_1^{(L)}, \dots, z_N^{(L)} \in \mathbb{R}^D$,
- optionally a special token: $z_{\text{cls}}^{(L)} \in \mathbb{R}^D$.

Need a single vector representation to feed into a classifier head.

- If using $[CLS]$, take its final representation:

$$y = \text{Head}(z_{\text{cls}}^{(L)}).$$

- Alternatively, pool patch tokens (mean pooling):

$$y = \text{Head}\left(\frac{1}{N} \sum_{i=1}^N z_i^{(L)}\right).$$

ViT became the new SOTA

- Shows a pure Transformer can work well for vision when trained at scale.
- Provides a simple, modular backbone:
 - easy to scale (depth/width),
 - integrates conditioning naturally (tokens / attention),
 - transferable across tasks.

Diffusion Transformer

Replace U-Net with a Transformer

- DiT uses a Transformer to predict noise (or v in v prediction) at each diffusion step.
- Pipeline:
 - 1 Patchify $x_t \in \mathbb{R}^{H \times W \times C}$ into tokens.
 - 2 Add positional encodings + timestep embedding (and optional class/text conditioning).
 - 3 Run Transformer blocks to produce token outputs.
 - 4 Unpatchify to image-shaped prediction $\hat{\epsilon}$ (or $\hat{v}$).

DiT Block Design

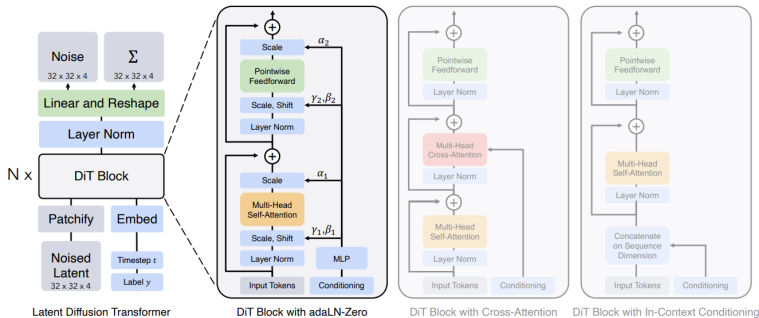


Figure 3: **The Diffusion Transformer (DiT) architecture.** *Left:* We train conditional latent DiT models. The input latent is decomposed into patches and processed by several DiT blocks. *Right:* Details of our DiT blocks. We experiment with variants of standard transformer blocks that incorporate conditioning via adaptive layer norm, cross-attention and extra input tokens. Adaptive layer norm works best.

Peebles, W. and Xie, S., 2023. Scalable diffusion models with transformers. In Proceedings of the IEEE/CVF international conference on computer vision (pp. 4195-4205).



AdaLN

Recall LN:

$$\text{LN}(h) = \gamma \odot \frac{h - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta, \quad \mu = \frac{1}{D} \sum_{i=1}^D h_i, \quad \sigma^2 = \frac{1}{D} \sum_{i=1}^D (h_i - \mu)^2,$$

$\gamma, \beta \in \mathbb{R}^D$, $\epsilon > 0$, $\odot$ denotes elementwise multiplication.

Adaptive LayerNorm: scale/shift depend on a conditioning vector c (e.g., timestep e_t)

$$\text{AdaLN}(h; c) = \gamma(c) \odot \frac{h - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta(c), \quad \gamma(c), \beta(c) \in \mathbb{R}^D.$$

Typical parameterization (linear or MLP)

$$\gamma(c) = W_\gamma c, \quad \beta(c) = W_\beta c, \\ W_\gamma, W_\beta \in \mathbb{R}^{D \times d_c}, \quad c \in \mathbb{R}^{d_c}.$$

Diffusion needs the same network to **behave differently** at different noise levels. AdaLN provides a clean mechanism: the timestep embedding modulates every block by changing its normalization parameters, effectively giving each layer “knobs” controlled by timestep t .



Timestep conditioning

- Diffusion requires injecting t (and possibly c) into each block.
- Common pattern: adaptive layer norm / FiLM-style modulation:

$$\text{AdaLN}(h; t, c) = \gamma(t, c) \odot \frac{h - \mu}{\sigma} + \beta(t, c).$$

- Lets the same network behave differently at different noise levels.

α gate

- In DiT, conditioning (timestep t , label y , etc.) produces *AdaLN* parameters and a **residual gate** α for each sublayer.
- For self-attention (gate α_1) and the feed forward network (gate α_2):

$$\text{cond} \xrightarrow{\text{MLP}} (\gamma_1, \beta_1, \alpha_1, \gamma_2, \beta_2, \alpha_2).$$

Block equations

$$h \leftarrow h + \alpha_1 \cdot \text{SA}\left(\text{AdaLN}(h; \gamma_1, \beta_1)\right),$$

$$h \leftarrow h + \alpha_2 \cdot \text{FF}\left(\text{AdaLN}(h; \gamma_2, \beta_2)\right).$$

- **“Zero” init:** $\alpha_1, \alpha_2 \approx 0$ at initialization $\Rightarrow$ each block starts near identity.
- Training learns to *turn on* attention/MLP updates gradually, improving stability.

- **Scaling:** model capacity increases by depth/width like standard Transformers.
- **Global context:** attention can connect distant regions easily.
- **Conditioning-friendly:** tokens / adaptive normalization make conditioning simple.
- **Systems:** efficient training stacks for Transformers (parallelism, kernels, mixed precision).



Limitations

- **Attention scaling:** patch tokens $N = \frac{H}{P} \frac{W}{P}$, self-attention $\sim O(N^2 D) \Rightarrow$ high-res can be expensive without latents / efficient attention.
- **Weaker CNN inductive bias:** less built-in locality / multi-scale structure than U-Nets $\Rightarrow$ may need more data/compute for the same efficiency.
- **Memory/throughput:** attention + large activations can be heavy compared to conv U-Nets.
- **Often relies on latent space:** high-res practicality typically requires an autoencoder $\Rightarrow$ possible compression artifacts / loss of fine detail.

Further Reading

Becomes the new standard for diffusion and video models
WAN2.1

Discussion: Xiao, Z., Kreis, K. and Vahdat, A., 2021. Tackling the generative learning trilemma with denoising diffusion GANs. arXiv preprint arXiv:2112.07804.